

## Notebook: all\_algo\_comparisons/knn\_comparison.ipynb

[ --- MARKDOWN CELL --- ]

# K-Nearest Neighbors (KNN) - Algorithm Comparison

This notebook implements KNN from scratch and compares it with sklearn on Breast Cancer and Wine datasets.

[ --- CODE CELL --- ]

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_breast_cancer, load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

[ --- CODE CELL --- ]

```
np.random.seed(42)
```

[ --- MARKDOWN CELL --- ]

## KNN Implementation From Scratch

[ --- CODE CELL --- ]

```
class KNNScratch:
    """K-Nearest Neighbors classifier from scratch."""

    def __init__(self, k=3):
        self.k = k
        self.X_train = None
        self.y_train = None

    def fit(self, X, y):
        """Store training data."""
        self.X_train = np.asarray(X)
        self.y_train = np.asarray(y)
        return self

    def _euclidean_distance(self, a, b):
        """Compute Euclidean distance between two vectors."""
        return np.sqrt(np.sum((a - b) ** 2))

    def _predict_single(self, x):
        """Predict class for a single sample."""
        # Compute distances to all training samples
        distances = []
        for x_train in self.X_train:
            dist = self._euclidean_distance(x, x_train)
            distances.append(dist)

        # Get indices of k nearest neighbors
        k_indices = np.argsort(distances)[:self.k]
        k_labels = self.y_train[k_indices]

        # Majority vote
        counts = {}
        for label in k_labels:
            counts[label] = counts.get(label, 0) + 1
        return max(counts.items(), key=lambda x: x[1])[0]

    def predict(self, X):
        """Predict classes for multiple samples."""
        X = np.asarray(X)
```

```

        predictions = [self._predict_single(x) for x in X]
        return np.array(predictions)

    def score(self, X, y):
        """Return accuracy score."""
        y_pred = self.predict(X)
        return accuracy_score(y, y_pred)

```

### [ --- MARKDOWN CELL --- ]

## Dataset 1: Breast Cancer

### [ --- CODE CELL --- ]

```

# Load Breast Cancer dataset
data_bc = load_breast_cancer()
X_bc, y_bc = data_bc.data, data_bc.target
print(f"Breast Cancer Dataset:")
print(f"  Samples: {X_bc.shape[0]}, Features: {X_bc.shape[1]}")
print(f"  Classes: {data_bc.target_names}")

```

### [ --- CODE CELL --- ]

```

# Split and scale
X_train_bc, X_test_bc, y_train_bc, y_test_bc = train_test_split(
    X_bc, y_bc, test_size=0.2, random_state=42
)

scaler_bc = StandardScaler()
X_train_bc_scaled = scaler_bc.fit_transform(X_train_bc)
X_test_bc_scaled = scaler_bc.transform(X_test_bc)

```

### [ --- CODE CELL --- ]

```

# Train From Scratch KNN
knn_scratch_bc = KNNScratch(k=5)
knn_scratch_bc.fit(X_train_bc_scaled, y_train_bc)

# Train Sklearn KNN
knn_sklearn_bc = KNeighborsClassifier(n_neighbors=5)
knn_sklearn_bc.fit(X_train_bc_scaled, y_train_bc)

# Evaluate
acc_scratch_bc = knn_scratch_bc.score(X_test_bc_scaled, y_test_bc)
acc_sklearn_bc = knn_sklearn_bc.score(X_test_bc_scaled, y_test_bc)
print(f"Breast Cancer - Accuracy Comparison:")
print(f"  From Scratch: {acc_scratch_bc:.4f}")
print(f"  Sklearn:      {acc_sklearn_bc:.4f}")

```

### [ --- MARKDOWN CELL --- ]

## Dataset 2: Wine

### [ --- CODE CELL --- ]

```

# Load Wine dataset
data_wine = load_wine()
X_wine, y_wine = data_wine.data, data_wine.target
print(f"Wine Dataset:")
print(f"  Samples: {X_wine.shape[0]}, Features: {X_wine.shape[1]}")
print(f"  Classes: {data_wine.target_names}")

```

### [ --- CODE CELL --- ]

```

# Split and scale
X_train_wine, X_test_wine, y_train_wine, y_test_wine = train_test_split(
    X_wine, y_wine, test_size=0.2, random_state=42
)

```

```
)
scaler_wine = StandardScaler()
X_train_wine_scaled = scaler_wine.fit_transform(X_train_wine)
X_test_wine_scaled = scaler_wine.transform(X_test_wine)
```

### [ --- CODE CELL --- ]

```
# Train From Scratch KNN
knn_scratch_wine = KNNScratch(k=5)
knn_scratch_wine.fit(X_train_wine_scaled, y_train_wine)

# Train Sklearn KNN
knn_sklearn_wine = KNeighborsClassifier(n_neighbors=5)
knn_sklearn_wine.fit(X_train_wine_scaled, y_train_wine)

# Evaluate
acc_scratch_wine = knn_scratch_wine.score(X_test_wine_scaled, y_test_wine)
acc_sklearn_wine = knn_sklearn_wine.score(X_test_wine_scaled, y_test_wine)

print(f"Wine - Accuracy Comparison:")
print(f"  From Scratch: {acc_scratch_wine:.4f}")
print(f"  Sklearn:      {acc_sklearn_wine:.4f}")
```

### [ --- MARKDOWN CELL --- ]

## Comparison Visualization

### [ --- CODE CELL --- ]

```
# Create comparison bar chart
datasets = ['Breast Cancer', 'Wine']
scratch_scores = [acc_scratch_bc, acc_scratch_wine]
sklearn_scores = [acc_sklearn_bc, acc_sklearn_wine]

x = np.arange(len(datasets))
width = 0.35

fig, ax = plt.subplots(figsize=(10, 6))
bars1 = ax.bar(x - width/2, scratch_scores, width, label='From Scratch', color='steelblue')
bars2 = ax.bar(x + width/2, sklearn_scores, width, label='Sklearn', color='coral')

ax.set_ylabel('Accuracy')
ax.set_title('KNN: From Scratch vs Sklearn Comparison')
ax.set_xticks(x)
ax.set_xticklabels(datasets)
ax.legend()
ax.set_ylim(0.8, 1.05)
ax.grid(True, alpha=0.3, axis='y')

# Add value labels
for bar in bars1:
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
            f'{bar.get_height():.4f}', ha='center', va='bottom')
for bar in bars2:
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
            f'{bar.get_height():.4f}', ha='center', va='bottom')

plt.tight_layout()
plt.show()
```

### [ --- MARKDOWN CELL --- ]

## K Value Tuning

### [ --- CODE CELL --- ]

```
# Test different k values on Breast Cancer dataset
k_values = range(1, 21)
scratch_accs = []
sklearn_accs = []
```

```

for k in k_values:
    # From Scratch
    knn_s = KNNScratch(k=k)
    knn_s.fit(X_train_bc_scaled, y_train_bc)
    scratch_accs.append(knn_s.score(X_test_bc_scaled, y_test_bc))

    # Sklearn
    knn_sk = KNeighborsClassifier(n_neighbors=k)
    knn_sk.fit(X_train_bc_scaled, y_train_bc)
    sklearn_accs.append(knn_sk.score(X_test_bc_scaled, y_test_bc))

plt.figure(figsize=(12, 6))
plt.plot(k_values, scratch_accs, 'o-', label='From Scratch', markersize=8)
plt.plot(k_values, sklearn_accs, 's-', label='Sklearn', markersize=8)
plt.xlabel('K (Number of Neighbors)')
plt.ylabel('Accuracy')
plt.title('KNN Accuracy vs K Value (Breast Cancer Dataset)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.xticks(k_values)
plt.tight_layout()
plt.show()

```

[ --- MARKDOWN CELL --- ]

## ## Conclusion

The KNN implementation from scratch achieves comparable accuracy to sklearn's implementation.

- Both implementations show similar performance across different datasets
- The from-scratch version helps understand the algorithm mechanics
- Optimal k value varies by dataset and should be tuned using cross-validation